INDIAN INSTITUTE OF TECHNOLOGY DELHI

Numerical Analysis Undergraduate Research Project

Reproducing and Evaluating Finite-Difference Error Estimation in Physics-Informed Neural Networks

Author and Collaborators

Harshith N
Nilesh Kumar Verma
Devesh Saini

Course Coordinator

Jayati Sarkar

Date: October 2026

Course submission deadline: November 10, 2026

Project Repository: https://github.com/harsh-bitverse/Numerical-Insight-Lab

Primary Reference: Aleks Krasowski et al., *Building Trust in PINNs: Error Estimation through Finite Difference Methods*, arXiv:2603.15526 (2026).

Upstream Code: https://github.com/aleks-krasowski/pinn-fdm-error-estimation
Pinned snapshot commit: `460d66bc2a59720883fa6a06e8c5436db4f99020`.

Abstract

Physics-Informed Neural Networks (PINNs) provide a flexible framework for solving differential equations, but estimating their pointwise error without an analytical solution remains challenging. Krasowski et al. (2026) proposed a residual-based framework that estimates PINN error by numerically solving the associated PDE defect equation with finite-difference methods (FDM). This study reproduces selected findings from that work for the one-dimensional heat-equation benchmark and evaluates an extension to soft boundary constraints.

For the baseline configuration ($N_x = N_t = 64$, 10,000 Adam iterations, and float64 arithmetic), the measured RMS discrepancy of the residual-based estimator was 4.771×10^{-6}, compared with 1.613×10^{-4} for the direct-FDM estimator, corresponding to an advantage factor of approximately 33.80. A grid study over $N_x = N_t \in \{8, 16, 32, 64, 128, 256\}$ showed approximately second-order convergence along the tested coupled spatial-temporal refinement path; for $N_x \geq 32$, the fitted orders were $p_{\text{res}} = 1.982$ and $p_{\text{fdm}} = 1.990$. Because space and time were refined together, these results do not isolate their individual convergence orders.

We also evaluated the homogeneous defect solver on a soft-constrained PINN. In this tested setup, ignoring the nonzero initial and boundary defects led to a discrepancy that stalled near 10^{-2}. We therefore implemented an inhomogeneous defect formulation that incorporates those defects without requiring the analytical solution. For the heat-equation experiments, this formulation exhibited approximately second-order convergence and an estimator advantage of 85.67 at $N_x = N_t = 64$. Finally, a boundary-penalty sweep over $\lambda_{\text{bc}} \in \{1, 10, 100, 1000\}$, performed with one random seed, produced fitted convergence orders close to two for the inhomogeneous estimator across the tested settings. These observations are empirical and limited to the benchmark, grids, and training configurations studied; they do not establish universal convergence across penalty regimes or other PDEs.

1. Introduction & Research Question

Physics-Informed Neural Networks (PINNs) approximate the solution of differential equations by training neural networks to minimize a multi-objective loss consisting of PDE residuals, boundary penalties, and initial condition mismatches (Raissi et al., 2019). Despite empirical success, certifying the accuracy of a trained PINN remains difficult. While the PDE residual $R(x,t) = \mathcal{D}[\widehat{u}](x,t)$ is readily computable via automatic differentiation, a small residual does not guarantee small pointwise error $e(x,t) = u(x,t) - \widehat{u}(x,t)$, particularly for stiff, advection-dominated, or ill-conditioned systems.

To bridge this gap, Krasowski et al. (2026) proposed solving the linearized defect equation:

$$\mathcal{L}[e] = -R(x,t)$$

using classical finite-difference methods (specifically Crank-Nicolson time stepping). The authors claim that solving for the error perturbation e directly on a coarse grid is orders of magnitude more accurate than comparing the PINN against a direct numerical solution of the physical state u.

Research Questions

In this undergraduate numerical analysis research project at IIT Delhi, we investigate: 1. **Reproducibility:** Does the authors' implementation execute as reported, and do the baseline metrics match published claims and reference datasets? 2. **Grid-Resolution Behavior:** How does the finite-difference grid spacing $h = \Delta x = \Delta t$ govern the estimator's accuracy, what are the empirical versus asymptotic convergence orders, and does the advantage persist across untrained networks? 3. **Boundary Condition Extension:** How does the residual error estimator perform when the PINN uses soft penalty boundary constraints rather than exact hard constraints? Can the defect framework be generalized to inhomogeneous boundary errors without requiring analytical ground truth?

2. Background & Relevant Numerical Analysis Concepts

2.1 The Principle of Defect Correction

In classical numerical analysis, defect correction (or residual correction) is an established technique where an approximate solution $\widehat{u}$ is iteratively refined by solving an auxiliary equation driven by the residual defect (Stetter, 1978; Hairer et al., 1993). In the PINN context, rather than using the

defect solution to correct the network weights, Krasowski et al. repurpose the defect equation as an *a posteriori* error estimator.

2.2 Crank-Nicolson Finite Difference Scheme

For parabolic equations of the form $u_t = \alpha u_{xx}$, the Crank-Nicolson scheme is a standard implicit, two-level finite difference method. Denoting the spatial grid spacing by $\Delta x = 1/(N_x - 1)$ and time step by $\Delta t = 1/(N_t - 1)$, the second-order central difference matrix for ∂_{xx} is denoted A_{xx}. The scheme averages spatial derivatives at time steps n and $n+1$:

$$\frac{u^{n+1} - u^n}{\Delta t} = \frac{\alpha}{2} A_{xx} \left(u^{n+1} + u^n\right)$$

which rearranges into the linear system:

$$\left(I - \frac{\alpha \Delta t}{2} A_{xx}\right) u^{n+1} = \left(I + \frac{\alpha \Delta t}{2} A_{xx}\right) u^n$$

The Crank-Nicolson scheme is unconditionally A-stable with theoretical local truncation error $\mathcal{O}(\Delta x^2 + \Delta t^2)$.

3. Methodology & Governing Equations

3.1 1D Heat Equation Specification

We evaluate the standard 1D linear heat equation on the spatio-temporal domain $\Omega \times [0, T] = [0, 1] \times [0, 1]$:

$$\frac{\partial u}{\partial t} - \alpha \frac{\partial^2 u}{\partial x^2} = 0, \quad x \in (0, 1),\ t \in (0, 1]$$

with physical parameters $\alpha = 0.05$, sinusoidal initial condition $u(x, 0) = \sin(2\pi x)$, and homogeneous Dirichlet boundary conditions $u(0, t) = u(1, t) = 0$. The analytical ground truth is:

$$u_{\text{true}}(x, t) = e^{-4\pi^2 \alpha t} \sin(2\pi x)$$

3.2 Error Defect Derivation & Implementation Signs

Let $\hat{u}(x, t)$ denote the continuous neural network prediction. The forward PDE residual is defined in the implementation (`heat_1d.py`) as:

$$R(x, t) := \frac{\partial \hat{u}}{\partial t} - \alpha \frac{\partial^2 \hat{u}}{\partial x^2}$$

Defining the true pointwise signed error as $e(x, t) := u_{\text{true}}(x, t) - \hat{u}(x, t)$, we subtract the equations:

$$\left(\frac{\partial u_{\text{true}}}{\partial t} - \alpha \frac{\partial^2 u_{\text{true}}}{\partial x^2}\right) - \left(\frac{\partial \hat{u}}{\partial t} - \alpha \frac{\partial^2 \hat{u}}{\partial x^2}\right) = 0 - R(x, t)$$

$$\implies \frac{\partial e}{\partial t} - \alpha \frac{\partial^2 e}{\partial x^2} = -R(x, t)$$

Discretizing via Crank-Nicolson on grid nodes (x_i, t_n):

$$\left(I - \frac{\alpha \Delta t}{2} A_{xx}\right) e^{n+1} = \left(I + \frac{\alpha \Delta t}{2} A_{xx}\right) e^n - \frac{\Delta t}{2} \left(R(x, t_n) + R(x, t_{n+1})\right)$$

Notice the strictly negative sign on the residual forcing term $-0.5\Delta t(R^n + R^{n+1})$, which matches `upstream/pinn_error/core/fdm.py` line 523 and our implementation in `scripts/run_boundary_extension.py` line 85.

3.3 Metric Definitions & Norm Distinctions

To avoid ambiguity, all error metrics across this paper follow the standardized definitions below:

1. **True Pointwise Error:** $e_{\text{true}}(x,t) := u_{\text{true}}(x,t) - \widehat{u}(x,t)$.
2. **Residual Estimator Discrepancy:** $\Delta_{\text{res}}(x,t) := e_{\text{true}}(x,t) - e_{\text{res}}(x,t)$, where e_{res} is the numerical solution of the defect equation.
3. **Direct-FDM Estimator Discrepancy:** $\Delta_{\text{FDM}}(x,t) := e_{\text{true}}(x,t) - e_{\text{FDM}}(x,t)$, where $e_{\text{FDM}} := u_{\text{FDM}} - \widehat{u}$ is the standard numerical benchmark.
4. **Discrete Root Mean Square (RMS) Norm:** Evaluated over all $N = N_x N_t$ grid points:

$$\|V\|_{\text{RMS}} := \sqrt{\frac{1}{N_x N_t} \sum_{n=0}^{N_t-1} \sum_{i=0}^{N_x-1} V(x_i,t_n)^2}$$

   Important Distinction: The discrete RMS norm treats every node with uniform weight $1/N$, including boundary nodes $x \in \{0,1\}$ and $t \in \{0,1\}$. It is an empirical root-mean-square error over sample points, not continuous L^2 quadrature on $[0,1] \times [0,1]$.
5. **Continuous Composite Trapezoidal Quadrature:**

$$\|V\|_{\text{trapz}} := \sqrt{\sum_{n=0}^{N_t-1} \sum_{i=0}^{N_x-1} w_n w_i \, V(x_i,t_n)^2 \, \Delta x \, \Delta t}$$

   where each one-dimensional weight w_i or w_n equals $1/2$ at a boundary node and 1 at an interior node; their product therefore gives weight $1/4$ at a corner. Because hard-constrained PINNs enforce near-zero boundary error ($< 10^{-15}$), boundary discrepancy contributions are negligible, and discrete RMS and trapezoidal quadrature yield nearly indistinguishable advantage factors ($33.80\times$ vs $33.87\times$).
6. **Advantage Factor:**

$$\text{Advantage Factor} := \frac{\|\Delta_{\text{FDM}}\|_{\text{RMS}}}{\|\Delta_{\text{res}}\|_{\text{RMS}}} = \frac{\|u_{\text{true}} - u_{\text{FDM}}\|_{\text{RMS}}}{\|e_{\text{true}} - e_{\text{res}}\|_{\text{RMS}}}$$

   Crucial Clarification: The advantage factor strictly measures the relative accuracy of the two *error estimators* (Δ_{FDM} vs Δ_{res}) in approximating the true error e_{true}. It does *not* compare the true PINN prediction error against the FDM solution error.

   Key Theoretical Insight (Algebraic Cancellation): Expanding the direct-FDM discrepancy:

$$\Delta_{\text{FDM}} = (u_{\text{true}} - \widehat{u}) - (u_{\text{FDM}} - \widehat{u}) \equiv u_{\text{true}} - u_{\text{FDM}}$$

   The neural network prediction $\widehat{u}$ cancels algebraically. Consequently, Δ_{FDM} depends *exclusively* on the truncation error of the numerical PDE solver and is completely independent of the PINN weights, training iterations, and random initialization seed.

4. Experimental Setup & Verification Protocol

- **Software Environment:** Isolated Python 3.12.10 virtual environment (`.venv`), PyTorch 2.14.1, DeepXDE 1.15.0, NumPy 1.26.4, SciPy 1.15.2, Polars 1.26.0, Matplotlib 3.10.1 on Windows 11.
- **Precision & Hardware:** Standardized to double precision (`float64`) and CPU execution (`set_default_device("cpu")`).
- **Neural Architecture:** 4-layer fully connected MLP `[2, 20, 20, 20, 1]` with `tanh` activations, Glorot normal weight initialization, and Adam optimization (lr $= 10^{-3}$).
- **Sampling:** 10,000 domain collocation points sampled uniformly across $[0,1] \times [0,1]$.
- **Model Checkpointing:** Unique SHA-256 configuration hashes cache trained networks in `upstream/pinn-fdm-error-estimation/model_zoo/` and `results/models/`, ensuring reproducible model checkpoints across grid sweeps.

5. Baseline Reproduction Results

The baseline configuration reproduced the published 1D Heat Equation benchmark ($N_x = N_t = 64$, 10,000 iterations, seed 42). Table 1 compares our measured results against the upstream reference dataset (`all_experiments_results_float64.csv`, seeds 0–9).

Table 1: Baseline Reproduction Verification ($N_x = N_t = 64$, Seed 42)

Metric	Upstream Reported / Reference Dataset	Our Measured Run	Absolute Departure	Verification Status
True PINN Error $\|e_{\text{true}}\|_{\text{RMS}}$	$(2.6 \pm 5.7) \times 10^{-4}$ (Seeds 0–9)	1.051475×10^{-4}	Within 1.0σ	**VERIFIED FROM CODE/DATA**
Residual Discrepancy $\|\Delta_{\text{res}}\|_{\text{RMS}}$	$(5.976 \pm 0.991) \times 10^{-6}$ (Seeds 0–9)	4.771048×10^{-6}	$z = -1.21$	**VERIFIED FROM CODE/DATA**
Direct-FDM Discrepancy $\|\Delta_{\text{FDM}}\|_{\text{RMS}}$	$1.6126004189332567 \times 10^{-4}$ ($\sigma = 0$)	$1.6126004189332567 \times 10^{-4}$	$\mathbf{1.83 \times 10^{-15}}$	**NEAR-MACHINE-PRECISION NUMERICAL AGREEMENT**
Advantage Factor	$\sim 27\times$ (mean)	**33.80**×	Consistent with seed	**VERIFIED FROM CODE/DATA**
FDM Solve Time	1.6 ± 0.3 ms	4.0 ms	< 5 ms	**VERIFIED FROM CODE/DATA**
Residual Integration Time	106 ± 8 ms	116.7 ms	~ 0.1 s	**VERIFIED FROM CODE/DATA**

As established by the algebraic cancellation property, $\|\Delta_{\text{FDM}}\|_{\text{RMS}}$ matches the authors' reference CSV across all 10 seeds down to 1.83×10^{-15} (near machine precision), providing evidence that our linear solver (`scipy.sparse.linalg.spsolve`) and spatial stencil ($L = \alpha A_{xx}$) execute with high numerical consistency relative to the upstream reference. Importantly, this near-machine-precision agreement reflects deterministic FDM linear algebra on matching grids; in contrast, PINN training involves stochastic optimization where reproduction is properly assessed by distributional consistency across random initializations.

6. Grid-Resolution & Convergence Analysis

To evaluate how grid refinement impacts estimator accuracy, we executed a controlled sweep across $N_x = N_t \in \{8, 16, 32, 64, 128, 256\}$ under fixed frozen network weights. We compared two distinct regimes: Well-trained (10,000 iterations) versus Random initialization (0 iterations). All raw metrics are preserved in `results/grid_study.csv` and plotted in Figure 1.

Table 2: Grid-Resolution Study Metrics & Convergence Orders

Regime	N	h	True error	Residual discrepancy	FDM discrepancy	Advantage	p_{res}	p_{fdm}
Well-Trained	8	0.142857	8.528×10^{-5}	4.367×10^{-4}	1.217×10^{-2}	**27.87x**	—	—
	16	0.066667	9.865×10^{-5}	7.688×10^{-5}	2.749×10^{-3}	**35.76x**	**2.2791**	**1.9522**
	32	0.032258	1.039×10^{-4}	1.920×10^{-5}	6.581×10^{-4}	**34.29x**	**1.9114**	**1.9694**
	64	0.015873	1.051×10^{-4}	4.771×10^{-6}	1.613×10^{-4}	**33.80x**	**1.9631**	**1.9832**
	128	0.007874	1.057×10^{-4}	1.185×10^{-6}	3.993×10^{-5}	**33.71x**	**1.9873**	**1.9913**
	256	0.003922	1.060×10^{-4}	2.948×10^{-7}	9.934×10^{-6}	**33.69x**	**1.9950**	**1.9956**
Random Init	8	0.142857	4.065×10^{-1}	1.621×10^{-2}	1.217×10^{-2}	**0.75x**	—	—
	16	0.066667	4.205×10^{-1}	3.586×10^{-3}	2.749×10^{-3}	**0.77x**	**1.9793**	**1.9522**
	32	0.032258	4.274×10^{-1}	8.462×10^{-4}	6.581×10^{-4}	**0.78x**	**1.9893**	**1.9694**
	64	0.015873	4.309×10^{-1}	2.057×10^{-4}	1.613×10^{-4}	**0.78x**	**1.9946**	**1.9832**
	128	0.007874	4.326×10^{-1}	5.071×10^{-5}	3.993×10^{-5}	**0.79x**	**1.9973**	**1.9913**
	256	0.003922	4.334×10^{-1}	1.259×10^{-5}	9.934×10^{-6}	**0.79x**	**1.9986**	**1.9956**

Figure 1. Grid-resolution convergence study. *The corresponding image file was not included in the supplied source files.* Estimator discrepancy $\|\Delta\|_{\text{RMS}}$ versus grid spacing $\Delta x = \Delta t$ across spatial resolutions $N_x \in [8, 256]$. Shaded envelopes represent Sample Mean $\pm$ 1 Sample Standard Deviation ($n = 10$ random seeds from `all_experiments_results_float64.csv`). Solid markers denote our measured runs (Seed 42).*

Analysis of Grid Study Findings:

1. **Asymptotic Convergence Order:** Over the asymptotic range $N_x \in [32, 256]$, linear regression of $\ln(\|\Delta\|)$ against $\ln(h)$ yields asymptotic slopes of $p_{\textbf{res}} = 1.9822$ and $p_{\textbf{fdm}} = 1.9901$ for well-trained PINNs, and $p_{\textbf{res}} = 1.9969$ and $p_{\textbf{fdm}} = 1.9901$ for random initialization. This is consistent with second-order Crank-Nicolson convergence along the tested coupled refinement path ($N_x = N_t, \Delta x = \Delta t = h$). *Methodological Clarification:* Because Δx and Δt were refined simultaneously, this study establishes the coupled rate $\mathcal{O}(\Delta x^2 + \Delta t^2) = \mathcal{O}(h^2)$ rather than independently separating spatial from temporal convergence orders.
2. **Pre-asymptotic Coarse Grid Effect:** At $N_x = 8$, the spatial grid is too coarse to resolve residual curvature, leading to an apparent transition order of $p \approx 2.28$ when refining from $N_x = 8 \to 16$.
3. **Persistent Advantage in Well-Trained Models:** For $N_x \geq 16$, the residual estimator maintains an almost constant advantage of $33.7\times$ **to** $35.8\times$ over direct FDM. Because the defect equation solves directly for the perturbation e, it bypasses the severe cancellation error ($u_{\text{FDM}} - \widehat{u}$) inherent in direct simulation.
4. **Behavior under Random Initialization:** For untrained networks, e_{res} converges at the same second-order rate ($p \approx 2.00$). However, because the untrained residual is large and spatially irregular, e_{res} has $\approx 25\%$ higher error than e_{FDM} (Advantage Factor $\approx 0.78\times$), faithfully reproducing Figure 3(c) of the primary paper.

7. The Boundary-Condition Extension

7.1 Motivation: The Hard Constraint Assumption

The primary paper restricts its formulation to exact hard constraints:

$$\widehat{u}_{\text{hard}}(x,t) = \sin(2\pi x) + t\,x(1-x)\,\mathcal{N}(x,t;\theta)$$

which enforces $e(x,0) = e(0,t) = e(1,t) \equiv 0$ to floating-point precision. In realistic applications with complex geometries, practitioners widely use **soft constraints** via penalty loss functions.

7.2 Failure of the Upstream Solver on Soft PINNs

In `upstream/pinn_error/core/fdm.py`, the solver explicitly forces $e(x,0) = 0$ and $e(0,t) = e(1,t) = 0$. When a soft-constrained PINN is passed to this unmodified solver, boundary departure ($\approx 1.4\%$) is forcibly ignored. As shown in Table 3, the naive estimator's error stalls at $\|\Delta_{\text{naive}}\|_{\text{RMS}} \approx 1.05 \times 10^{-2}$ across all grids ($p = 0.00$), performing up to $262\times$ **worse** than direct FDM (advantage factor $0.0038\times$ at $N_x = 128$).

7.3 Mathematical Resolution: Inhomogeneous Defect Solver

We extend the defect formulation to inhomogeneous boundary and initial conditions:

$$\frac{\partial e}{\partial t} - \alpha \frac{\partial^2 e}{\partial x^2} = -R(x,t)$$

$$e(x,0) = u_0(x) - \widehat{u}(x,0), \quad e(0,t) = 0 - \widehat{u}(0,t), \quad e(1,t) = 0 - \widehat{u}(1,t)$$

Foundational Property: Because $u_0(x) = \sin(2\pi x)$ and boundary values $g = 0$ are given by the problem definition, this formulation requires **zero analytical ground truth**. We implemented `InhomogeneousFDMSolverHeatEq`, which injects boundary errors into the Crank-Nicolson RHS at each time step.

Table 3: Hard vs. Soft BC Comparative Study Results

Model / solver	N_x	True error	Max boundary error	Estimator discrepancy	FDM discrepancy	Advantage	Order p
Hard BC	16	9.865×10^{-5}	$< 10^{-15}$	7.688×10^{-5}	2.749×10^{-3}	**35.76x**	—
Hard BC	32	1.039×10^{-4}	$< 10^{-15}$	1.920×10^{-5}	6.581×10^{-4}	**34.29x**	$p = 1.91$
Hard BC	64	1.051×10^{-4}	$< 10^{-15}$	4.771×10^{-6}	1.613×10^{-4}	**33.80x**	$p = 1.96$
Hard BC	128	1.057×10^{-4}	$< 10^{-15}$	1.185×10^{-6}	3.993×10^{-5}	**33.71x**	$p = 1.99$
Soft BC (inhom.)	16	1.219×10^{-2}	1.385×10^{-2}	3.243×10^{-5}	2.749×10^{-3}	**84.76x**	—

Model / solver	N_x	True error	Max boundary error	Estimator discrepancy	FDM discrepancy	Advantage	Order p
Soft BC (inhom.)	32	1.224×10^{-2}	1.386×10^{-2}	7.630×10^{-6}	6.581×10^{-4}	**86.25x**	$p = 1.99$
Soft BC (inhom.)	64	1.227×10^{-2}	1.386×10^{-2}	1.882×10^{-6}	1.613×10^{-4}	**85.67x**	$p = 1.97$
Soft BC (inhom.)	128	1.228×10^{-2}	1.386×10^{-2}	4.668×10^{-7}	3.993×10^{-5}	**85.53x**	$p = 1.99$
Soft BC (naive)	16	1.219×10^{-2}	1.385×10^{-2}	1.052×10^{-2}	2.749×10^{-3}	**0.26x**	—
Soft BC (naive)	32	1.224×10^{-2}	1.386×10^{-2}	1.049×10^{-2}	6.581×10^{-4}	**0.06x**	$p = 0.00$ **(Stalled)**
Soft BC (naive)	64	1.227×10^{-2}	1.386×10^{-2}	1.048×10^{-2}	1.613×10^{-4}	**0.02x**	$p = 0.00$ **(Stalled)**
Soft BC (naive)	128	1.228×10^{-2}	1.386×10^{-2}	1.047×10^{-2}	3.993×10^{-5}	**0.0038x**	$p = 0.00$ **(Stalled)**

Figure 2. Boundary-condition extension comparison. *The corresponding image file was not included in the supplied source files.* Comparative evaluation of Hard vs. Soft BC error estimation. (a) Spatial error profile slice at $t = 0.5$. (b) Boundary error time series $e(0, t)$ along $x = 0$. (c) Estimator discrepancy versus grid spacing $\Delta x = \Delta t$. (d) Advantage factor across grid resolutions.*

7.4 Extension Findings:

1. **Restoration of Second-Order Convergence:** `InhomogeneousFDMSolverHeatEq` restores observed second-order convergence ($p = 1.99 \pm 0.01$), tracking both interior error and boundary drift faithfully.
2. **Elevated Advantage Factor (**$85.67\times$ **at** $N_x = 64$**):** Because soft PINNs have higher error magnitude ($\approx 1.23 \times 10^{-2}$), direct FDM incurs large cancellation error ($u_{\text{FDM}} - \widehat{u}$). In contrast, the defect solver integrates the smooth residual R, achieving an advantage of $85.67\times$ at $N_x = 64$ (and between $84.8\times$ **and** $86.3\times$ across resolutions) over direct FDM.

7.5 Parametric Sensitivity to Boundary Penalty Weight ($\lambda_{\mathbf{bc}} \in \{1, 10, 100, 1000\}$)

To evaluate how boundary loss weighting influences the defect estimator, we executed a controlled parametric sweep across $\lambda_{\text{bc}} \in \{1, 10, 100, 1000\}$ on `seed=42`, protected by SHA-256 cryptographic cache fingerprinting (`scripts/model_cache.py`). Table 4 summarizes the empirical findings evaluated across grid resolutions $N_x = N_t \in \{16, 32, 64\}$.

Table 4: Boundary Penalty Sweep Results (64×64**, Seed 42)**

Penalty λ_{bc}	True PINN error	FDM discrepancy	Inhom. discrepancy	Inhom. order	Inhom. advantage	Naive discrepancy	Naive order	Naive advantage
$\lambda_{\mathbf{bc}} = 1$	1.147×10^{-3}	1.613×10^{-4}	1.737×10^{-6}	2.00	92.83×	1.653×10^{-3}	0.00	0.098×
$\lambda_{\mathbf{bc}} = 10$	1.110×10^{-3}	1.613×10^{-4}	5.750×10^{-6}	1.99	28.05×	9.682×10^{-4}	0.16	0.167×
$\lambda_{\mathbf{bc}} = 100$	6.354×10^{-3}	1.613×10^{-4}	1.700×10^{-5}	2.00	9.48×	6.099×10^{-3}	0.07	0.026×
$\lambda_{\mathbf{bc}} = 1000$	3.518×10^{-1}	1.613×10^{-4}	1.613×10^{-4}	1.98	1.00×	3.516×10^{-1}	0.00	0.00046×

Figure 3. Boundary-penalty sweep comparison. *The corresponding image file was not included in the supplied source files.* Controlled boundary-penalty sweep across $\lambda_{\text{bc}} \in \{1, 10, 100, 1000\}$. (a) Inhomogeneous estimator convergence ($p \approx 2.00$). (b) Naive homogeneous estimator error stalling ($p \approx 0.00$). (c) Trade-off between boundary conformity and interior solution error. (d) Estimator advantage factor η over direct FDM.*

Observations from the Penalty Sweep:

1. **Observed Convergence Across Tested Penalties:** In this single-seed sweep, the inhomogeneous defect solver produced fitted orders close to two ($p = 1.98$–2.00) for the four tested penalty weights. The fitted orders are consistent with the expected second-order discretization behavior in these experiments; the single-seed sweep does not establish complete decoupling from neural training dynamics.
2. **Persistence of Naive Estimator Stagnation:** The homogeneous solver showed little or no convergence ($p = 0.00$–0.16) for the tested values of λ_{bc}. The observed behavior is consistent

with the effect of nonzero boundary departures ($\delta_b \sim 5.5 \times 10^{-4}$), although the experiments did not directly isolate the mechanism.

3. **Loss Imbalance and Multi-Objective Trade-Off:** At $\lambda_{\text{bc}} = 1000$, the run showed a substantial loss trade-off: the initial-condition loss was $\mathcal{L}_{\text{ic}} \approx 0.498$, and the true PINN error increased from 1.15×10^{-3} to 0.3518. This observation is consistent with multi-objective gradient-conflict issues discussed by Wang et al. (2021), but parameter-level gradients were not logged, so the mechanism was not directly verified.
4. **Estimator Decoupling & Parity at $\lambda_{\text{bc}} = 1000$:** Even when the PINN fails as a solver ($e_{\text{true}} \approx 0.3518$), the inhomogeneous estimator accurately predicts the true error field with discrepancy 1.613×10^{-4} ($p = 1.98$). At this limit, the defect equation becomes mathematically equivalent to the original heat equation, causing the defect truncation error to match direct FDM truncation error to 4 significant digits ($\eta \approx 1.00\times$).
5. **Zero Information Leakage:** Both defect solvers use zero analytical ground truth; boundary and initial defects are evaluated strictly against prescribed problem functions $g(t) = 0, u_0(x) = \sin(2\pi x)$.

8. Discussion, Limitations & Threats to Validity

8.1 What the Results Establish

- For the tested one-dimensional heat-equation configurations, the residual-based FDM estimator provided an accurate approximation of PINN error, with an estimator-discrepancy advantage over direct FDM of $33.80\times$ (hard BC) to $92.83\times$ (soft BC, $\lambda_{\text{bc}} = 1$) in estimator discrepancy on the baseline 64×64 grid.
- The measured discrepancies exhibited approximately $\mathcal{O}(h^2)$ convergence for both trained and randomly initialized networks along the tested coupled refinement trajectory.
- In the tested heat-equation setup, accounting for the initial and boundary defects restored the observed convergence behavior. These experiments do not establish that this is the only possible remedy for soft-constrained PINNs.

8.2 What the Results Do Not Establish

- **Not a Certified Rigorous Bound:** e_{res} is an *approximation* of error, not an analytical upper bound. While e_{bound} provides guaranteed bounds via semigroup theory, it lacks spatial resolution.
- **Coupled Refinement Trajectory:** The grid study tested coupled diagonal refinement ($N_x = N_t$). While this demonstrates the overall $\mathcal{O}(h^2)$ asymptotic behavior of the coupled Crank-Nicolson discretization, it does not experimentally isolate spatial discretization error from temporal stepping error.
- **Limited Scope of Extension:** Experiments were conducted on the 1D Heat Equation. The derivation may extend to higher dimensions and other linear PDEs, but this was not tested here; nonlinear systems (e.g., Navier–Stokes) would require additional treatment, such as Newton–Raphson defect linearization.
- **Stiffness and Advection Limits:** In advection-dominated or stiff regimes, standard second-order central differences can develop spurious oscillations unless upwinding or stabilized schemes are employed.
- **Single-Seed Sweep Scope:** The penalty sweep was executed on a single random initialization (`seed=42`). While the results are mathematically consistent and reproducible via cached checkpoints, multi-seed statistical distributions were not gathered for the penalty study.
- **Gradient Telemetry:** Parameter-level backpropagation gradients were not logged to disk during training; the gradient pathology mechanism is inferred from observed loss component trajectories and literature consensus (Wang et al., 2021).

9. Conclusion

This study reproduced selected baseline findings for the one-dimensional heat-equation benchmark and evaluated a finite-difference error estimator using the reported reference data and our measured runs. At $N_x = N_t = 64$, the residual-based estimator discrepancy was 4.771×10^{-6}, compared with 1.613×10^{-4} for the direct-FDM discrepancy, giving an advantage factor of 33.80. Along the tested coupled grid-refinement path, the measured discrepancies exhibited approximately second-order

convergence; because spatial and temporal resolutions were refined simultaneously, their individual contributions were not isolated.

For the soft-constrained heat-equation experiment, the unmodified homogeneous solver did not account for nonzero initial and boundary defects, and its discrepancy stalled near 10^{-2}. Incorporating the prescribed initial and boundary defects in an inhomogeneous defect formulation restored the observed convergence behavior in this setup. The resulting estimator advantage was 85.67 at $N_x = N_t = 64$. In the boundary-penalty sweep, the fitted order for the inhomogeneous estimator remained close to two for the four tested penalty weights. However, that sweep used a single random seed, and the results should be interpreted as observations for the tested configurations rather than a universal guarantee. The experiments were limited to the one-dimensional heat equation and do not establish performance for other PDEs or independently determine spatial and temporal convergence rates.

References

1. Aleks Krasowski, Nick Heilenkötter, and Jan S. Hesthaven. *Building Trust in PINNs: Error Estimation through Finite Difference Methods*. arXiv preprint arXiv:2603.15526, 2026.
2. Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*. Journal of Computational Physics, 378:686–707, 2019.
3. Lu Lu, Xuhui Meng, Zhiping Mao, and George E. Karniadakis. *DeepXDE: A deep learning library for solving differential equations*. SIAM Review, 63(1):208–228, 2021.
4. Hans J. Stetter. *The defect correction approach in defect estimation and defect improvement for differential equations*. Computing, 20(3):219–232, 1978.
5. Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I: Nonstiff Problems*. Springer-Verlag, Berlin, 1993.
6. A. Hillebrecht and B. Unger. *Certified error bounds for physics-informed neural networks*. arXiv preprint arXiv:2308.14480, 2023.
7. S. Wang, Y. Teng, and P. Perdikaris. *Understanding and mitigating gradient pathologies in physics-informed neural networks*. SIAM Journal on Scientific Computing, 43(5):A3055–A3081, 2021.

Appendix: Reproducibility Commands

To reproduce all results in a clean environment:

```
# 1. Activate environment
.venv\Scripts\activate

# 2. Run Reproducibility & Mathematical Invariant Tests
pytest -v tests/test_reproducibility.py

# 3. Re-run Grid Study (uses cached weights, runs in ~3 s)
python scripts/run_grid_study.py

# 4. Re-run Boundary Condition Extension (uses cached weights, runs in ~4 s)
python scripts/run_boundary_extension.py

# 5. Re-run Controlled Penalty Weight Sweep (uses cached weights, runs in ~8 s)
python scripts/run_penalty_sweep.py
```

All raw data are preserved in `results/grid_study.csv`, `results/boundary_extension.csv`, and `results/penalty_sweep/heat_penalty_sweep.csv`.